CECS 326 – Operating Systems
Homework Assignment 2

Due Date

TBD

Objective

The objective of this assignment is to gain hands-on experience implementing the classical **Producer–Consumer Problem** using **POSIX threads** and **POSIX semaphores** in Linux.

Students will design a bounded circular buffer shared by multiple producer and consumer threads while ensuring correct synchronization, mutual exclusion, and deadlock-free execution.

Learning Outcomes

After completing this assignment, students should be able to:

- Understand race conditions.
- Implement mutual exclusion using semaphores.
- Synchronize multiple threads.
- Implement a bounded circular buffer.
- Prevent buffer overflow and underflow.
- Use POSIX threads in Linux.
- Use POSIX semaphores.
- Design concurrent programs safely.

Problem Description

Implement a program that simulates a warehouse.

Several **Producer** threads generate products.
Several **Consumer** threads remove products from the warehouse.
The warehouse contains a bounded circular buffer.
A producer must wait if the buffer is full.
A consumer must wait if the buffer is empty.
Only one thread may modify the buffer at any time.

Programming Requirements

Part 1 – Thread Creation (15 points)

Create

- 3 Producer threads
- 3 Consumer threads

Each thread must display:

- Thread ID
- Thread type
- Start message

Example:

```
Producer 2 started.
Thread ID: 1402847399

Consumer 1 started.
Thread ID: 1402847200
```

Part 2 – Circular Buffer (20 points)

Implement a bounded circular buffer.

Requirements:

- Buffer size = 10
- Use an integer array
- Maintain:

```
in
out
count
```

Do NOT use STL queue or vector.

Part 3 – Synchronization (35 points)

Use POSIX semaphores.

Create the following semaphores:

```
mutex
empty
full
```

Initialize them correctly.

Implement the Producer and Consumer exactly according to the bounded-buffer algorithm discussed in class.

Your implementation must prevent:

- Race conditions
- Buffer overflow
- Buffer underflow

Busy waiting is NOT allowed.

Part 4 – Product Generation (10 points)

Each producer generates exactly 20 products.

Each product contains:

```
Producer ID
Product Number
```

Example:

```
Producer 2 produced Product 14
```

Generate a random delay

```
sleep(rand() % 3);
```

between productions.

Part 5 – Product Consumption (10 points)

Consumers remove products until every product has been consumed.

Display:

```
Consumer 1 consumed Product 14 from Producer 2
```

Generate random delays between consumptions.

Part 6 – Final Statistics (10 points)

After all threads terminate, the main thread prints:

```
============================
Execution Summary
============================

Products Produced : 60
Products Consumed : 60

Producer 1 : 20
Producer 2 : 20
Producer 3 : 20
```

```
Consumer 1 : XX
Consumer 2 : XX
Consumer 3 : XX

Buffer Empty
Program completed successfully.
```

Restrictions

You may NOT use:

- std::mutex
- std::lock_guard
- std::condition_variable
- Busy waiting
- Global locks other than the required semaphores
- STL queue
- STL vector

You MUST use:

- pthread_create()
- pthread_join()
- sem_init()
- sem_wait()
- sem_post()
- Linux
- C++
- g++

Deliverables

Submit a single ZIP file named exactly:

`lastname_HW02.zip`

Inside the ZIP file include:

- producer_consumer.cpp

- Makefile
- README.pdf
- Screenshot(s) showing successful compilation and execution

The README must include:

- Student Name
- Student ID
- Compilation instructions
- Execution instructions

Grading Rubric

Criterion	**Points**
Correct creation of all threads	10
Correct implementation of circular buffer	20
Correct semaphore synchronization	30
No race conditions	10
Correct producer behavior	10
Correct consumer behavior	10
Correct final statistics	5
Program compiles successfully using g++ without modification	3
Code readability, indentation, comments, and organization	2
Total	**100**

Submission Penalties

Violation	**Deduction**
ZIP file name does not follow lastname_HW02.zip format	–5
Source file missing	–20
Makefile missing	–10
README missing	–5
Screenshot missing	–5
Program does not compile	Up to –40
Program crashes during execution	Up to –30

Submitted files not inside a ZIP archive	–10
Incorrect or extra files causing grading issues	Up to –5

Reflection Questions (Not Graded)

The following questions are intended to reinforce concepts covered in class. You are encouraged to research these topics independently. These questions may appear on quizzes and exams.

1. What is a race condition?
2. Why is mutual exclusion necessary?
3. What is the purpose of the `mutex` semaphore?
4. Why are both the `empty` and `full` semaphores required?
5. What would happen if the `mutex` semaphore were removed?
6. What is busy waiting, and why is it inefficient?
7. Explain the difference between a binary semaphore and a counting semaphore.
8. Why is a circular buffer preferred over a linear buffer?
9. What would happen if there were more consumers than producers?
10. Explain why this implementation prevents both buffer overflow and buffer underflow.

Extra Credit (+10 Points)

Modify your program so that the number of producers, consumers, and the buffer size are specified using command-line arguments.

Example:

```
./producer_consumer 5 4 20
```

where:

- 5 = number of producers
- 4 = number of consumers
- 20 = buffer size

Your program should dynamically allocate the buffer and create the specified number of threads while maintaining correct synchronization.

Academic Integrity

All submitted work must be your own. You may discuss concepts with classmates, but the implementation, debugging, and submitted source code must be completed individually.

Students are responsible for ensuring that their programs compile and execute correctly before submission.